Experiment Title : Practical 1

Aim:

- Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem. Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

Theory:

The Two Water Jug Problem and The Missionaries and Cannibals problem is a classical Artificial Intelligence problem that demonstrates state-space representation and systematic problem solving.

In two water jug problem fixed capacities are given, and the goal is to measure an exact target amount of water using only basic operations such as filling a jug, emptying a jug, and pouring water from one jug to another. Each state is represented as (X, Y), where X and Y denote the water in jug X and jug Y. Starting from (0, 0), new states are generated by applying valid operations until the target state is reached.

In missionaries and cannibals problem each state is represented as a tuple (m,c,b), where m and c denote the number of missionaries and cannibals on the left bank, and b indicates the boat's position. The objective is to safely transfer all individuals to the right bank without ever allowing cannibals to outnumber missionaries on either side. The solution uses Breadth First Search (BFS) to systematically explore all valid states, ensuring the shortest sequence of legal boat crossings is found while respecting capacity and safety constraints.

Algorithm / Methodology :

Two water jug problem algorithm:

1. Program takes CapX , CapY and target state from user.
2. Set initial state to (0,0).
3. Display the action states from menu to fill, empty and pour water from one jug to another for both jugs.
4. Based on user choice the current state gets updated after every choice.
5. After every update check if target is reached else loop until target state is met.
6. Once target is reached terminate the program.

Three missionaries and three cannibals problem algorithm:

1. Set total Missionaries (M) and Cannibals (C) and initial state (M, C, 0) with goal (0, 0, 1).
2. Display the initial state.
3. Initialize a queue and visited list to track states.
4. While goal not reached, remove the first state from the queue.
5. Generate all valid next states by moving 1 or 2 people according to boat position and safety rules.
6. Add unvisited valid states to the queue and repeat.
7. Once goal is reached, display the solution path and terminate.

Dataset Description:

In two jug problem dataset consists of all possible valid states.

Jug capacities (CapX, CapY)

Current state (X, Y)

Target state entered by the user

In missionaries and cannibals problem only valid states are allowed.

Missionaries and cannibals can be 0,1,2 or 3.

And boat can only be 0 or 1.

Implementation:

Code of two water jug:

```
global CapX,CapY,X,Y


def TwoWaterJug(CapX, CapY, target):

 X,Y=0,0

 while([X,Y] != target):


   print('Action to be taken:\n 1. fill X \n 2. fill Y \n 3. pour X into Y\n 4. pour Y into X\n
5. Empty X \n 6. Empty Y')

   user=int(input())

   match user:

     case 1:
```

```
  X = CapX
  Y = Y
  print(X,Y)
case 2:
  Y = CapY
  X = X
  print(X,Y)
case 3:
  if(Y+X<=CapY):
   Y=Y+X
   X=0
  else:
    X=X-(CapY-Y)
    Y=CapY
  print(X,Y)

case 4:
 if(X+Y<=CapX):
   X=X+Y
   Y=0
 else:
   X=CapX
   Y=Y-(CapX-X)
 print(X,Y)

case 5:
 X=0
 Y=Y
 print(X,Y)
```

```
    case 6:
      Y=0
      X=X
      print(X,Y)
    case _:
      print("Invalid input")

def main():
  print('enter the capacity of the jugs')
  CapX, CapY = map(int,input().split())
  print("enter the target")
  target = list(map(int, input().split()))
  print(target)
  TwoWaterJug(CapX, CapY, target)
  print("Target Achieved!!!!\n YOU WON!!!")
main()
```

Code of missionaries and cannibals:

```
C = 3
M = 3

# state (m,c,b)
#b=0 is leftbank b=1 is rightbank
start = (3,3,0)
goal = (0,0,1)

def isValid(state): #checking if desired state is valid
  m, c, b = state
```

```
  if m<0 or c<0 or m>3 or c>3:
    return False
  if c>m and m>0: #left bank condition
    return False
  if (3-c)>(3-m) and (3-m)>0: #right bank condition
    return False
  return True

def next_move(state): #generating possible next states
  m, c, b = state
  move = [(2,0), (0,2), (1,1), (1,0), (0,1)]
  next_state = []
  for dm,dc in move:
    if b == 0:
      new_state = (m-dm, c-dc,1)
    else:
      new_state = (m+dm, c+dc,0)

    if isValid(new_state):
      next_state.append(new_state) #checking which state is valid and then appending it

  return next_state

def missonaries_cannibals():
  queue =[]
  state = start
  print(state)

  queue.append((state, [state]))
```

```
  visited = [state]

  while start != goal:
    #boat constarint only 2 can be carried.
    #boat takes 2 people
    #check sanity if it allowed
    state, path = queue.pop(0)

    if state == goal:
      print("Solution found:")
      for s in path:
        print(s)
      return

    for next in next_move(state):
      if next not in visited:
        queue.append((next, path + [next]))
        visited.append(next)

  print("No solution found")


missonaries_cannibals()
```

Output/Result:

```
enter the capacity of the jugs
3 5
enter the target
0 4
[0, 4]
Action to be taken:
 1. fill X
 2. fill Y
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
2
0 5
Action to be taken:
 1. fill X
 2. fill Y
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
4
3 5
Action to be taken:
 1. fill X
 2. fill Y
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
6
3 0
```

```
6
3 0
Action to be taken:
 1. fill X
 2. fill Y
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
3
0 3
Action to be taken:
 1. fill X
 2. fill Y
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
1
3 3
Action to be taken:
 1. fill X
 2. fill Y
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
3
1 5
Action to be taken:
 1. fill X
 2. fill Y
```

```
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
6
1 0
Action to be taken:
 1. fill X
 2. fill Y
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
3
0 1
Action to be taken:
 1. fill X
 2. fill Y
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
1
3 1
Action to be taken:
 1. fill X
 2. fill Y
 3. pour X into Y
 4. pour Y into X
 5. Empty X
 6. Empty Y
3
0 4
```

```
(3, 3, 0)
Solution found:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
```

Result analysis:

It is goal driven game which makes user reach for goal. Repeated valid states helps reach target eventually.

Only safe and valid states are allowed in missionaries and cannibals problem. No unsafe moves occur in the game.

Conclusion:

This is the use of BFS, state representation and constraint check.

Both the problems are using state-space search.

This problem can be solved for different capacity of jugs and number of missionaries and cannibals.